Submitted by-

Esha Ghosh

A1-B1-07

EXPERIMENT TITLE

EXPERIMENT 01: Water Jug Problem and Missionaries & Cannibals Problem

Aim / Objective

- Solving 2 water jug Problem: Write a program to solve the 2-water jug problem by manually providing moves from the user.
- Write a program to implement Missionaries and Cannibals problem. Constraints: three missionaries and three cannibals must cross a river using a boat which can carry at most two people, under the constraint that, for both banks, if there are missionaries present on the bank, they cannot be outnumbered by cannibals (if they were, the cannibals would eat the missionaries). The boat cannot cross the river by itself with no people on board.

Theory

The "Water Jug Problem" and the "Missionaries and Cannibals Problem" are very well known problems in Artificial Intelligence that are used to demonstrate and study – state space representation, constraint satisfaction and problem solving using different search algorithms.

Water Jug Problem: In this problem, we have 2 water jugs with fixed water capacities and a goal water measure (goal state) that we want to achieve. The problem solution requires applying a fixed set of transitions which are – empty a jug, completely fill a jug and pour the water from one jug to another. Each state has been denoted by a pair of numbers, (X, Y), which represents the current level of water in each jug. The process continues until it reaches the goal

state. This problem assertion clearly explains how deterministic transitions occur and how it helps in constraint satisfaction problems.

Missionaries and Cannibals Problem: This is a classic river crossing problem that illustrates state space search and constraint satisfaction. In this problem, there are 3 missionaries and 3 cannibals; and we have to move them across a river using a boat, provided condition that the boat can carry a maximum of 2 people. The most important constraint is that, at any point of time, on either of the banks, missionaries must not be fewer in number than cannibals, otherwise the cannibals would eat up the missionaries. BFS is used to search all valid states while ensuring that there are no violations of the above-mentioned constraints. BFS is guaranteed to find the shortest and most secure way to the final state.

Algorithm / Methodology (Step-wise, numbered)

A. Water Jug Problem

1. **Start**
2. Initialize jug **X = 0** (capacity 3 liters) and jug **Y = 0** (capacity 5 liters).
3. Repeat the following steps until the final state **(X = 0 and Y = 4)** is reached:
    - Display menu options to the user:
        1. Fill jug X
        2. Fill jug Y
        3. Empty jug X
        4. Empty jug Y
        5. Pour water from X to Y
        6. Pour water from Y to X
4. Read the user's choice.
5. Perform the selected operation:
    - Filling sets the jug to its maximum capacity.

- Emptying sets the jug to zero.
- Pouring transfers water until either the source is empty or the destination is full.

6. After every move, display the current state **(X, Y)**.
7. Continue until the goal state is achieved.
8. Display the final state.
9. **Stop**

B. Missionaries and Cannibals Problem

1. **Start**
2. Define total missionaries = 3 and total cannibals = 3.
3. Represent each state as **(m, c, b)** where:
   - *m* = missionaries on left bank
   - *c* = cannibals on left bank
   - *b* = boat position (0 = left, 1 = right)
4. Define valid boat moves allowing 1 or 2 people.
5. Check state validity:
   - Missionaries are never outnumbered by cannibals on either bank.
6. Use **Breadth-First Search (BFS)**:
   - Start from initial state **(3, 3, 0)**.
   - Generate all valid neighboring states.
   - Store visited states to avoid repetition.
7. Continue BFS until goal state **(0, 0, 1)** is reached.
8. Backtrack using parent pointers to obtain the solution path.
9. Display the sequence of states and corresponding moves. **Stop**

Dataset Description (Source, attributes, size)

2–Water Jug Problem

- Source: User input during program execution
- Attributes:
  - X: Amount of water in jug X (0–3 liters)
  - Y: Amount of water in jug Y (0–5 liters)
- Dataset Size: Dynamic; depends on user moves (maximum 24 possible states)

Missionaries and Cannibals Problem

- Source: Program-generated states based on problem constraints
- Attributes:
  - m: Missionaries on left bank (0–3)
  - c: Cannibals on left bank (0–3)
  - b: Boat position (0 = left, 1 = right)
- Dataset Size: 32 possible states, fewer valid due to safety rules

Implementation

1. **Water Jug Problem**

```
import sys   # Importing sys module (not strictly needed here, but kept)

# Initialize the two jugs

# X has capacity 3 liters

# Y has capacity 5 liters

X = 0

Y = 0
```

```
choice = -1  # Variable to store user choice

# Loop continues until the goal state (0,4) is reached
while not (X == 0 and Y == 4):

    # Take user input for operation
    choice = int(input("Enter choice: "))

    # 1. Fill jug X completely
    if choice == 1:
        print("Fill X")
        X = 3
        print(X, Y)

    # 2. Fill jug Y completely
    elif choice == 2:
        print("Fill Y")
        Y = 5
        print(X, Y)

    # 3. Empty jug X
    elif choice == 3:
        print("Empty X")
        X = 0
        print(X, Y)

    # 4. Empty jug Y
    elif choice == 4:
```

```
        print("Empty Y")
        Y = 0
        print(X, Y)

    # 5. Pour water from jug X to jug Y
    elif choice == 5:
        print("Pour from X to Y")
        space = 5 - Y           # Available space in jug Y
        pour = min(X, space)    # Amount that can be poured
        X = X - pour
        Y = Y + pour
        print(X, Y)

    # 6. Pour water from jug Y to jug X
    elif choice == 6:
        print("Pour from Y to X")
        space = 3 - X           # Available space in jug X
        pour = min(Y, space)    # Amount that can be poured
        X = X + pour
        Y = Y - pour
        print(X, Y)

    # Invalid choice handling
    else:
        print("Invalid Choice")
        break

# Final goal state output
```

```
print("Final State:")
print(X, Y)
```

2. Missionaries and Cannibals Problem

```
from collections import deque    # deque is used for efficient BFS queue operations
# Total number of missionaries and cannibals
TOTAL_M = 3
TOTAL_C = 3


def is_valid(m, c):
    """
    Check whether a state is valid.
    m = missionaries on left bank
    c = cannibals on left bank
    """

    # Check boundary conditions
    if m < 0 or c < 0 or m > TOTAL_M or c > TOTAL_C:
        return False

    # LEFT BANK condition:
    # If missionaries are present, they must not be outnumbered by cannibals
    left_ok = (m == 0) or (m >= c)

    # RIGHT BANK counts
    mr = TOTAL_M - m   # missionaries on right bank
    cr = TOTAL_C - c   # cannibals on right bank
```

```
    # RIGHT BANK condition:
    # If missionaries are present, they must not be outnumbered
    right_ok = (mr == 0) or (mr >= cr)

    return left_ok and right_ok

# All possible valid boat moves (boat can carry 1 or 2 people)
# (missionaries, cannibals)
MOVES = [
    (2, 0),  # two missionaries
    (0, 2),  # two cannibals
    (1, 1),  # one missionary and one cannibal
    (1, 0),  # one missionary
    (0, 1)   # one cannibal
]

def neighbors(state):
    """
    Generate all valid neighboring states from the current state.
    state = (missionaries_left, cannibals_left, boat_side)
    boat_side: 0 = left bank, 1 = right bank
    """
    m, c, b = state
    result = []

    for dm, dc in MOVES:
        if b == 0:
            # Boat on LEFT bank → move people to RIGHT bank
```

```
            nm = m - dm
            nc = c - dc
            nb = 1
        else:
            # Boat on RIGHT bank → bring people to LEFT bank
            nm = m + dm
            nc = c + dc
            nb = 0

        # Add the new state only if it is valid
        if is_valid(nm, nc):
            result.append((nm, nc, nb))

    return result

def bfs(start, goal):
    """
    Breadth-First Search to find the shortest path
    from start state to goal state.
    """
    q = deque([start])          # BFS queue
    parent = {start: None}      # To reconstruct the solution path

    while q:
        cur = q.popleft()

        # Goal reached
        if cur == goal:
```

```
            path = []
            while cur is not None:
                path.append(cur)
                cur = parent[cur]
            path.reverse()
            return path

        # Explore neighboring states
        for nxt in neighbors(cur):
            if nxt not in parent:   # Avoid revisiting states
                parent[nxt] = cur
                q.append(nxt)

    return None  # No solution (theoretically won't happen here)

def print_solution(path):
    """
    Print the solution path and the corresponding moves.
    """
    if not path:
        print("No solution found.")
        return

    print("Solution path (m_left, c_left, boat_side):")
    for i, st in enumerate(path):
        print(f"{i:2d}: {st}")

    print("\nMoves:")
```

```
    for i in range(len(path) - 1):
        m1, c1, b1 = path[i]
        m2, c2, b2 = path[i + 1]

        # Number of people moved is the difference on left bank
        moved_m = abs(m2 - m1)
        moved_c = abs(c2 - c1)

        direction = "Left -> Right" if b1 == 0 else "Right -> Left"
        print(f"{i:2d}: {direction}  (M={moved_m}, C={moved_c})")

# Driver code
if __name__ == "__main__":
    # Initial state: all on left bank, boat on left
    start = (3, 3, 0)

    # Goal state: all on right bank, boat on right
    goal = (0, 0, 1)

    path = bfs(start, goal)
    print_solution(path)
```

Output / Result

1) Water Jug Problem

```
Enter choice: 2
Fill Y
0 5
Enter choice: 6
Pour from Y to X
3 2
Enter choice: 3
Empty X
0 2
Enter choice: 6
Pour from Y to X
2 0
Enter choice: 2
Fill Y
2 5
Enter choice: 6
Pour from Y to X
3 4
Enter choice: 3
Empty X
0 4
Final State:
0 4
```

2) Missionaries and Cannibals Problem

```
...
    Solution path (m_left, c_left, boat_side):
     0: (3, 3, 0)
     1: (3, 1, 1)
     2: (3, 2, 0)
     3: (3, 0, 1)
     4: (3, 1, 0)
     5: (1, 1, 1)
     6: (2, 2, 0)
     7: (0, 2, 1)
     8: (0, 3, 0)
     9: (0, 1, 1)
    10: (1, 1, 0)
    11: (0, 0, 1)

    Moves:
     0: Left -> Right  (M=0, C=2)
     1: Right -> Left  (M=0, C=1)
     2: Left -> Right  (M=0, C=2)
     3: Right -> Left  (M=0, C=1)
     4: Left -> Right  (M=2, C=0)
     5: Right -> Left  (M=1, C=1)
     6: Left -> Right  (M=2, C=0)
     7: Right -> Left  (M=0, C=1)
     8: Left -> Right  (M=0, C=2)
     9: Right -> Left  (M=1, C=0)
    10: Left -> Right  (M=1, C=1)
```

Result Analysis

The Water Jug program successfully reaches the desired goal state by applying valid user-defined operations such as filling, emptying, and pouring water between jugs. Each move updates the jug levels correctly, helping visualize state transitions clearly.

The Missionaries and Cannibals program finds a safe sequence of crossings using Breadth-First Search. It ensures that missionaries are never outnumbered by cannibals on either bank. The BFS approach guarantees a valid and optimal solution path. Overall, both programs demonstrate effective state-space exploration and constraint handling.

Conclusion

The Water Jug Problem demonstrates how a goal state can be achieved through valid state transitions using simple operations. The Missionaries and Cannibals problem shows the application of Breadth-First Search to solve a constrained state-space problem safely. Both programs help in understanding fundamental AI concepts such as state representation, constraints, and systematic problem solving. Thus, the objectives of the experiment are successfully achieved.